

UCS301 Data Structures

Lab Assignment 1

Q1 — Menu Driven Array Operations

Algorithm:

1. Start.
 2. Initialize the array and set $n = 0$.
 3. Display the menu:
 - ☐ CREATE
 - ☐ DISPLAY
 - ☐ INSERT
 - ☐ DELETE
 - ☐ LINEAR SEARCH
 - ☐ EXIT
 4. Read the user's choice.
 5. If choice is **CREATE**:
 - ☐ Read n .
 - ☐ Input n elements into the array.
 - ☐ Display "Array created successfully."
 6. If choice is **DISPLAY**:
 - ☐ Check if $n == 0$.
 - ☐ If empty, display the appropriate message.
 - ☐ Otherwise, traverse and display all elements.
 7. If choice is **INSERT**:
 - ☐ Check whether the array is full.
 - ☐ Read position and value.
 - ☐ Shift elements one position to the right.
 - ☐ Insert the value at $pos - 1$.
 - ☐ Increment n .
 8. If choice is **DELETE**:
 - ☐ Check whether the array is empty.
 - ☐ Read the position.
 - ☐ Shift elements after that position one position to the left.
 - ☐ Decrement n .
 9. If choice is **LINEAR SEARCH**:
 - ☐ Read the key element.
 - ☐ Compare it with every array element.
 - ☐ If found, display its position and index.
 - ☐ Otherwise display "Element not found."
 10. If choice is **EXIT**, terminate the program.
 11. Repeat the menu until EXIT is selected.
 12. Stop.
-

main.c

Run

Output



```
1
2 #include <stdio.h>
3 #define MAX 100
4 int arr[MAX];
5 int n = 0;
6 void createArray();
7 void displayArray();
8 void insertElement();
9 void deleteElement();
10 void linearSearch();
11
12 int main() {
13     int choice;
14
15     do {
16         printf("\n—MENU—\n");
17         printf("1. CREATE\n");
18         printf("2. DISPLAY\n");
19         printf("3. INSERT\n");
20         printf("4. DELETE\n");
21         printf("5. LINEAR SEARCH\n");
22         printf("6. EXIT\n");
23         printf("Enter your choice: ");
24         scanf("%d", &choice);
25
26         switch (choice) {
```

```
—MENU—
1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT
Enter your choice: 1
Enter number of elements (max 100): 3
Enter 3 elements:
Element[0]: 1
Element[1]: 2
Element[2]: 3
Array created successfully.
```

```
—MENU—
1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT
Enter your choice:
```

```
26         switch (choice) {
27             case 1: createArray(); break;
28             case 2: displayArray(); break;
29             case 3: insertElement(); break;
30             case 4: deleteElement(); break;
31             case 5: linearSearch(); break;
32             case 6: printf("Exiting program. Goodbye!\n");
                    break;
33             default: printf("Invalid choice! Please try
                        again.\n");
34         }
35     } while (choice != 6);
36
37     return 0;
38 }
39 void createArray() {
40     printf("Enter number of elements (max %d): ", MAX);
41     scanf("%d", &n);
42
43     if (n <= 0 || n > MAX) {
44         printf("Invalid size!\n");
45         n = 0;
46         return;
47     }
48
49     printf("Enter %d elements:\n", n);
```

```
50     for (int i = 0; i < n; i++) {
51         printf("Element[%d]: ", i);
52         scanf("%d", &arr[i]);
53     }
54     printf("Array created successfully.\n");
55 }
56
57
58 void displayArray() {
59     if (n == 0) {
60         printf("Array is empty. Please create it first\n\n");
61         return;
62     }
63     printf("Array elements: ");
64     for (int i = 0; i < n; i++) {
65         printf("%d ", arr[i]);
66     }
67     printf("\n");
68 }
69
70
71 void insertElement() {
72     int pos, value;
73
74     if (n >= MAX) {
75         printf("Array is full! Cannot insert.\n");
76     }
```

```

74     if (n > MAX) {
75         printf("Array is full. Cannot insert.\n");
76         return;
77     }
78
79     printf("Enter position to insert (1 to %d): ", n + 1);
80     scanf("%d", &pos);
81
82     if (pos < 1 || pos > n + 1) {
83         printf("Invalid position!\n");
84         return;
85     }
86
87     printf("Enter value to insert: ");
88     scanf("%d", &value);
89
90     for (int i = n; i >= pos; i--) {
91         arr[i] = arr[i - 1];
92     }
93     arr[pos - 1] = value;
94     n++;
95
96     printf("Element inserted successfully.\n");
97 }
98
99
100 void deleteElement() {
101     int pos;
102

```

```
103     if (n == 0) {
104         printf("Array is empty! Cannot delete.\n");
105         return;
106     }
107
108     printf("Enter position to delete (1 to %d): ", n);
109     scanf("%d", &pos);
110
111     if (pos < 1 || pos > n) {
112         printf("Invalid position!\n");
113         return;
114     }
115
116     for (int i = pos - 1; i < n - 1; i++) {
117         arr[i] = arr[i + 1];
118     }
119     n--;
120
121     printf("Element deleted successfully.\n");
122 }
123
124 void linearSearch() {
125     int key, found = 0;
126
127     if (n == 0) {
128         printf("Array is empty!\n");
129         return;
130     }
```

```
120
121     printf("Element deleted successfully.\n");
122 }
123
124 void linearSearch() {
125     int key, found = 0;
126
127     if (n == 0) {
128         printf("Array is empty!\n");
129         return;
130     }
131
132     printf("Enter element to search: ");
133     scanf("%d", &key);
134
135     for (int i = 0; i < n; i++) {
136         if (arr[i] == key) {
137             printf("Element found at position %d (index %d).  

138                 \n", i + 1, i);
139             found = 1;
140             break;
141         }
142
143     if (!found) {
144         printf("Element not found in the array.\n");
145     }
146 }
```

—MENU—

1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT

Enter your choice: 2

Array elements: 1 2 3

—MENU—

1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT

Enter your choice: 3

Enter position to insert (1 to 4): 2

Enter value to insert: 9

Element inserted successfully.

—MENU—

1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT

Enter your choice: 4

Enter position to delete (1 to 4): 2

Element deleted successfully.

—MENU—

1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT

Enter your choice: 5

Enter element to search: 1

Element found at position 1 (index 0).

```
1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT
Enter your choice: 5
Enter element to search: 1
Element found at position 1 (index 0).
```

```
—MENU—
1. CREATE
2. DISPLAY
3. INSERT
4. DELETE
5. LINEAR SEARCH
6. EXIT
Enter your choice: 6
Exiting program. Goodbye!
```

Q2. Q2 — Remove Duplicate Elements

Your code calls `removeDuplicates(arr, &`

Algorithm:

1. Start.
2. Read n and input the array elements.
3. Set $i = 0$.
4. Compare `arr[i]` with the elements following it.
5. Set $j = i + 1$.
6. If `arr[i] == arr[j]`:
 - Shift all elements from $j + 1$ one position to the left.
 - Decrease n by 1.
 - Continue checking the same j .
7. Otherwise, increment j .
8. Continue until all elements after `arr[i]` have been checked.

9. Increment i and repeat the process.
10. Display the array after removing duplicates.
11. Stop.

Programiz C Online Compiler

main.c

Run

```
1
2 #include <stdio.h>
3
4 #define MAX 100
5
6 void removeDuplicates(int arr[], int *n) {
7     for (int i = 0; i < *n; i++) {
8         for (int j = i + 1; j < *n; j++) {
9             if (arr[i] == arr[j]) {
10
11                 for (int k = j; k < *n - 1; k++) {
12                     arr[k] = arr[k + 1];
13                 }
14                 (*n)--;
15                 j--;
16             }
17         }
18     }
19 }
20
21 int main() {
22     int arr[MAX], n;
23
24     printf("Enter number of elements: ");
25     scanf("%d", &n);
26
27     if (n <= 0 || n > MAX) {
28         printf("Invalid size!\n");
29         return 1;
30     }
31 }
```

Output

Enter number of elements: 5
Enter 5 elements:
Element[0]: 1
Element[1]: 1
Element[2]: 2
Element[3]: 3
Element[4]: 3

Original array: 1 1 2 3 3

Array after removing duplicates: 1 2 3

=== Code Execution Successful ===

```

50     }
51
52     printf("Enter %d elements:\n", n);
53     for (int i = 0; i < n; i++) {
54         printf("Element[%d]: ", i);
55         scanf("%d", &arr[i]);
56     }
57
58     printf("\nOriginal array: ");
59     for (int i = 0; i < n; i++) {
60         printf("%d ", arr[i]);
61     }
62     printf("\n");
63
64     removeDuplicates(arr, &n);
65
66     printf("\nArray after removing duplicates: ");
67     for (int i = 0; i < n; i++) {
68         printf("%d ", arr[i]);
69     }
70     printf("\n");

```

Q3 — Rotate Array by K Positions

Left Rotation

1. Start.
2. Initialize the array and calculate n.
3. Read/set K.
4. If K == 0, return.
5. Reverse the first K elements.
6. Reverse the remaining n-K elements.
7. Reverse the complete array.
8. Display the left-rotated array.
9. Stop.

Right Rotation

1. Start.
2. Initialize the array and calculate n.
3. Read/set K.
4. If K == 0, return.

5. Reverse the last K elements using:
reverseArray(A, n-K, n-1).
6. Reverse the first n-K elements.
7. Reverse the complete array.
8. Display the right-rotated array.
9. Stop.

For your example, K = 2 gives left rotation 3 4 5 1 2 and right rotation 4 5 1 2 3.

main.cpp

Run

```
1  #include <iostream>
2  using namespace std;
3
4  void reverseArray(int A[], int start, int end) {
5      while (start < end) {
6          swap(A[start], A[end]);
7          start++;
8          end--;
9      }
10 }
11
12 void leftRotate(int A[], int n, int K) {
13     K = K % n;
14     if (K == 0) return;
15     reverseArray(A, 0, K - 1);
16     reverseArray(A, K, n - 1);
17     reverseArray(A, 0, n - 1);
18 }
19
20 void rightRotate(int A[], int n, int K) {
21     K = K % n;
22     if (K == 0) return;
```

```
20 void rightRotate(int A[], int n, int K) {
21     if (K == 0) return;
22     reverseArray(A, n - K, n - 1);
23     reverseArray(A, 0, n - K - 1);
24     reverseArray(A, 0, n - 1);
25 }
26
27
28 void printArray(int A[], int n) {
29     for (int i = 0; i < n; i++)
30         cout << A[i] << " ";
31     cout << endl;
32 }
33
34 int main() {
35     int A[] = {1, 2, 3, 4, 5};
36     int n = sizeof(A) / sizeof(A[0]);
37     int K = 2;
38
39     int leftArr[5], rightArr[5];
40     copy(A, A + n, leftArr);
41     copy(A, A + n, rightArr);
42 }
```

```
34 int main() {
35     int A[] = {1, 2, 3, 4, 5};
36     int n = sizeof(A) / sizeof(A[0]);
37     int K = 2;
38
39     int leftArr[5], rightArr[5];
40     copy(A, A + n, leftArr);
41     copy(A, A + n, rightArr);
42
43     cout << "Original array: ";
44     printArray(A, n);
45
46     leftRotate(leftArr, n, K);
47     cout << "Left rotation by " << K << ": ";
48     printArray(leftArr, n);
49
50     rightRotate(rightArr, n, K);
51     cout << "Right rotation by " << K << ": ";
52     printArray(rightArr, n);
53
54     return 0;
55 }
```



```
Original array: 1 2 3 4 5
```

```
Left rotation by 2: 3 4 5 1 2
```

```
Right rotation by 2: 4 5 1 2 3
```

```
=== Code Execution Successful ===
```

Q4. Q4(a) — Reverse Elements of an array

Algorithm:

1. Start.
2. Initialize the array.
3. Set start = 0.
4. Set end = n - 1.
5. Repeat while start < end:
 - Swap A[start] and A[end].
 - Increment start.
 - Decrement end.
6. Display the reversed array.
7. Stop.

For your code:

1 2 3 4 5 → 5 4 3 2 1.

```
1  #include <iostream>
2  using namespace std;
3
4  void reverseArray(int A[], int n) {
5      int start = 0, end = n - 1;
6      while (start < end) {
7          swap(A[start], A[end]);
8          start++;
9          end--;
10     }
11 }
12
13 void printArray(int A[], int n) {
14     for (int i = 0; i < n; i++)
15         cout << A[i] << " ";
16     cout << endl;
17 }
18
19 int main() {
20     int A[] = {1, 2, 3, 4, 5};
21     int n = sizeof(A) / sizeof(A[0]);
22
23     cout << "Original array: ";
24     printArray(A, n);
25
26     reverseArray(A, n);
```

```
27
28     cout << "Reversed array: ";
29     printArray(A, n);
30
31     return 0;
32 }
```

Output



```
Original array: 1 2 3 4 5
Reversed array: 5 4 3 2 1
```

Q4(b) — Matrix Multiplication

Algorithm:

1. Start.
2. Read the dimensions and elements of matrices A and B.
3. Check whether the number of columns of A equals the number of rows of B.
4. If not equal, display that multiplication is not possible.
5. Initialize the result matrix C with zero.
6. Use three nested loops:
 - Select each row i of A.
 - Select each column j of B.
 - Multiply $A[i][k]$ and $B[k][j]$ and add the result to $C[i][j]$.
7. Display matrix C.
8. Stop.

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int r1 = 2, c1 = 3;
6      int r2 = 3, c2 = 2;
7
8      int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
9      int B[3][2] = {{7, 8}, {9, 10}, {11, 12}};
10
11     if (c1 != r2) {
12         cout << "Matrix multiplication not possible" <<
13             endl;
14         return 0;
15     }
16
17     int C[2][2];
18
19     for (int i = 0; i < r1; i++) {
20         for (int j = 0; j < c2; j++) {
21             C[i][j] = 0;
22             for (int k = 0; k < c1; k++) {
```

```
4  int main() {  
16      int C[2][2];  
17  
18      for (int i = 0; i < r1; i++) {  
19          for (int j = 0; j < c2; j++) {  
20              C[i][j] = 0;  
21              for (int k = 0; k < c1; k++) {  
22                  C[i][j] += A[i][k] * B[k][j];  
23              }  
24          }  
25      }  
26  
27      cout << "Resultant Matrix:" << endl;  
28      for (int i = 0; i < r1; i++) {  
29          for (int j = 0; j < c2; j++) {  
30              cout << C[i][j] << " ";  
31          }  
32          cout << endl;  
33      }  
34  
35      return 0;  
36  }
```

Output



Resultant Matrix:

58 64

139 154

=== Code Execution Successful ===

Q4(c) — Transpose of a Matrix

Algorithm:

1. Start.
 2. Read the number of rows and columns.
 3. Input matrix A.
 4. Use two nested loops.
 5. For every element, assign:
 $T[j][i] = A[i][j]$.
 6. Display the transpose matrix T.
 7. Stop.
-

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int r = 2, c = 3;
6      int A[2][3] = {{1, 2, 3}, {4, 5, 6}};
7      int T[3][2];
8
9      for (int i = 0; i < r; i++) {
10         for (int j = 0; j < c; j++) {
11             T[j][i] = A[i][j];
12         }
13     }
14
15     cout << "Original Matrix:" << endl;
16     for (int i = 0; i < r; i++) {
17         for (int j = 0; j < c; j++) {
18             cout << A[i][j] << " ";
19         }
20         cout << endl;
21     }
22 }
```

```

22
23     cout << "Transposed Matrix:" << endl;
24     for (int i = 0; i < c; i++) {
25         for (int j = 0; j < r; j++) {
26             cout << T[i][j] << " ";
27         }
28         cout << endl;
29     }
30
31     return 0;
32 }

```

Output

Original Matrix:

1 2 3

4 5 6

Transposed Matrix:

1 4

2 5

3 6

Q5 — Sum of Every Row and Column

Algorithm:

1. Start.
2. Initialize $r = 3$ and $c = 3$.
3. Initialize the 3×3 matrix A.
4. Display the matrix using two nested loops.
5. For row sums:
 - Set rowSum = 0 for each row.
 - Traverse all columns of that row.
 - Add $A[i][j]$ to rowSum.
 - Display the sum of the row.
6. For column sums:
 - Set colSum = 0 for each column.
 - Traverse all rows of that column.

- Add $A[i][j]$ to colSum.
 - Display the sum of the column.
7. Stop.

main.cpp

Run

```
1  #include <iostream>
2  using namespace std;
3
4  int main() {
5      int r = 3, c = 3;
6      int A[3][3] = {
7          {1, 2, 3},
8          {4, 5, 6},
9          {7, 8, 9}
10     };
11
12     cout << "Matrix:" << endl;
13     for (int i = 0; i < r; i++) {
14         for (int j = 0; j < c; j++) {
15             cout << A[i][j] << " ";
16         }
17         cout << endl;
18     }
19
20     cout << "\nRow Sums:" << endl;
21     for (int i = 0; i < r; i++) {
22         int rowSum = 0;
23         for (int j = 0; j < c; j++) {
24             rowSum += A[i][j];
25         }
26         cout << "Row " << i << " sum = " << rowSum <<
            endl;
```



```
25     }
26     cout << "Row " << i << " sum = " << rowSum <<
    endl;
27 }
28
29 cout << "\nColumn Sums:" << endl;
30 for (int j = 0; j < c; j++) {
31     int colSum = 0;
32     for (int i = 0; i < r; i++) {
33         colSum += A[i][j];
34     }
35     cout << "Column " << j << " sum = " << colSum
    << endl;
36 }
37
38 return 0;
39 }
```



Matrix:

1 2 3

4 5 6

7 8 9

Row Sums:

Row 0 sum = 6

Row 1 sum = 15

Row 2 sum = 24

Column Sums:

Column 0 sum = 12

Column 1 sum = 15

Column 2 sum = 18